

Game Execution Flow & Architecture Documentation

This document outlines the execution flow, system loading order, and architectural structure of the Bevy game project.

1. High-Level Architecture

The game is built using the **Bevy Engine** (ECS architecture) and is structured around **Plugins** and **States**.

- **Entry Point:** `src/main.rs` initializes the `App`, adds default plugins, and registers custom game plugins.
- **State Management:** The game uses `GameState` to switch between major screens (Menu, Gameplay) and `GameplayState` for in-game states (Playing, Paused).
- **Asset Management:** Critical assets (textures, sounds) are preloaded at startup to ensure smooth transitions.

2. Initialization Flow (`src/main.rs`)

When the application starts, the following sequence occurs:

1. **Bevy Default Plugins:** Windowing, Rendering, Audio, Input, etc. are initialized.
 - *Configuration:* Window resolution is set to 1280x720 (resizable: false).
2. **Third-Party Plugins:** `AsepriteUltraPlugin` (Animation), `PixelCameraPlugin` (Camera).
3. **State Initialization:** `GameState::Loading` is set as the default state.
4. **Resource Initialization:**
 - `CurrentScene`: Tracks the active level.
 - `GameInputMap`: Defines player controls.
5. **Game Plugins:**
 - `LoadingPlugin`: Handles asset loading.
 - `MenuPlugin`: Handles the main menu.
 - `GameplayPlugin`: Handles the core game loop.
 - `SettingsPlugin`: Handles configuration.
 - `DespawnPlugin`: Handles entity cleanup on state changes.

3. State Transitions

GameState::Loading

- **OnEnter:** `LoadingState` starts loading `GameAssets`.
- **OnExit:** Transitions to `GameState::Menu` when assets are loaded.

GameState::Menu

- **OnEnter:** `menu_setup` spawns the UI camera and the Main Menu UI.
- **Update:**
 - `MenuSet::Interaction`: Handles button hovers/clicks and keyboard navigation.
 - `MenuSet::Actions`: Triggers state changes (e.g., clicking "Start" -> `GameState::Gameplay`).

- **OnExit:** `DespawnOnExit(GameState::Menu)` cleans up menu entities.

GameState::Settings

- **OnEnter:** `settings_setup` spawns the Settings UI.
- **Update:** Handles key remapping and navigation.
- **OnExit:** `DespawnOnExit(GameState::Settings)` cleans up settings entities.

GameState::Gameplay

- **OnEnter:**
 - `setup_gameplay`: Spawns the Player, NPC, and Camera.
 - `setup_hud`: Spawns the Health Bar and UI.
 - `load_map`: Spawns the Tiled Map (Bamboo Forest).
 - `start_playing`: Sets `GameState` to `Playing`.
- **Update:** The core gameplay loop runs (see Section 4).
- **OnExit:** `stop_gameplay` disables gameplay systems; `DespawnOnExit(GameState::Gameplay)` cleans up entities.

4. Gameplay Loop (`src/gameplay/mod.rs`)

The gameplay logic is organized into **System Sets** to ensure strict execution order within a frame. This prevents "off-by-one-frame" glitches (e.g., camera moving before the player moves).

Execution Order (Chain):

1. `GameplaySet::Input`

- `player_input`: Reads controller/keyboard input via `Leafwing`.
- `pause_input`: Checks for pause toggle.
- `sync_input_map`: Updates input mappings if changed.
- `npc_behavior`: Updates NPC AI decisions.
- `character_state_decision_system`: Determines the logical state (Idle, Run, Jump, Attack) based on input/physics.
- `sync_animation_system`: Syncs the logical state to the Aseprite animation component.

2. `GameplaySet::Physics`

- `physics`: Applies velocity, gravity, and handles collisions (AABB).
- `move_platforms`: Updates moving platform positions.

3. `GameplaySet::Camera`

- `camera_follow`: Smoothly interpolates the camera to follow the player (X-axis only, Y-axis fixed).
- `handle_camera_zoom`: Adjusts camera zoom level.

4. `GameplaySet::Animation`

- `flip_sprite_direction`: Flips sprite based on velocity.

5. `GameplaySet::UI`

- `hud_health`: Updates the health bar width/color based on player HP.
- `spawn_health_bars`: Spawns health bars for entities.
- `update_health_bars`: Updates health bar visuals.

5. Sub-Systems Breakdown

Map & Audio (`src/gameplay/map/`)

- **Loading**: `load_map` uses `bevy_ecs_tiled` to spawn the `.tmx` map.
- **Physics**: `GamePhysicsBackend` generates colliders from Tiled object layers.
- **Audio**: `spawn_map_audio` scans the map for objects with type "SoundEmitter" and spawns spatial audio sources (e.g., bird sounds) at their specific coordinates.

Mechanics (`src/gameplay/mechanics.rs`)

- **State Machine**: Manages character states (Idle, Walk, Jump, Fall, Attack, etc.).
- **Move Library**: Stores data for attacks (damage, frame data, knockback).
- **Combat**: `HitTracker` and `HitEvent` manage damage application and hitboxes.
- **Timers**: `JumpBuffer` and `CoyoteTime` for better platforming feel.

Camera System (`src/gameplay/camera.rs`)

- **Setup**: Spawns a `Camera2d` with `TiledParallaxCamera` support.
- **Scaling**: Uses `PixelZoom` to ensure pixel-perfect rendering.
- **Positioning**: Locked to `Z=50.0` to view background layers correctly while avoiding clipping.

UI System (`src/gameplay/ui.rs`)

- **Health Bars**: Dynamic health bars that follow entities.
- **HUD**: Heads-up display for player status.
- **Input Blocking**: `is_cursor_on_ui` prevents gameplay input when interacting with UI.

Combat System (`src/gameplay/combat.rs`)

- **Attack Config**: Defines properties for different attacks.
- **Hit Detection**: Manages hitboxes and damage application.
- **Projectiles**: Handles projectile spawning and movement.

6. Directory Structure Summary

```
src/  
├─ main.rs           # Entry point  
├─ assets/           # Asset loading (scene.rs)  
├─ common/           # Shared components (Input, Despawn)  
├─ states.rs         # State enums  
├─ loading.rs        # Loading screen logic  
├─ menu/             # Main menu logic  
├─ settings/         # Settings screen logic
```

```
|— gameplay/          # Core gameplay logic
|   |— mod.rs          # Gameplay plugin and system sets
|   |— camera.rs       # Camera logic
|   |— combat.rs       # Combat logic
|   |— debug.rs        # Debug visualization
|   |— game_over.rs    # Game over screen
|   |— input.rs        # Input handling
|   |— level.rs        # Level management (NPCs, platforms)
|   |— mechanics.rs    # Core mechanics (state machine)
|   |— mechanics_systems.rs # Mechanics systems
|   |— paused.rs       # Pause screen
|   |— player.rs       # Player spawning and logic
|   |— systems.rs      # General gameplay systems
|   |— ui.rs           # In-game UI
|   |— animation/      # Animation logic
|   |— map/            # Map loading and physics
```